

OCC: Software Developer

Computing Theory

COT512

Practical Assessment 7

2026



CTU
training solutions

Private
Tertiary
Education

Table of Contents

DECLARATION OF AUTHENTICITY.....	3
Pre-Assessment Agreement.....	4
QUALIFICATION AND ASSESSMENT DETAILS.....	8
INSTRUCTIONS.....	9
ASSESSMENT OUTCOME.....	9
QUESTION 1 (100).....	10
MARKING RUBRIC.....	12
MARK ALLOCATION.....	13
LEARNING OUTCOMES INTEGRATED QUESTIONS CROSS-MAPPING GRID.....	13

DECLARATION OF AUTHENTICITY

I Grant Samson (FULL NAME) hereby declare that the contents of this assessment PM-07 is entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work accessed from any **AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.**

Activity	Date

Signature: 

Date: 31/08/2026

Pre-Assessment Agreement

Assessment Preparation: Preparing the Candidate

Name of Candidate	Grant Samson	Date	17/08/2026
		Time	09:00
Name of Assessor	Mr NB Dube	Venue	Campus
How to prepare the candidate	Document Requirements	Agree (tick)	Action Required
Explain to the candidate why you are meeting and the purpose of the assessment.	Assessment Policy Assessment process	✓	
Discuss the assessment plan in detail.	Assessment strategy	✓	
Explain assessment process, show assessment instruments to candidate and describe assessment conditions.	Assessment instruments	✓	
Identify the role-players during assessment.	Assessors Moderator	✓	
Describe the evidence required to be declared competent.	Examples of evidence	✓	
Explain how evidence will be judged.	Mark allocation explained	✓	
Explain to the candidate how to prepare: Give candidate assessment task description.	Assessment task description	✓	
Confirm with the candidate what he/she should bring to the assessment.	Detailed briefing on exact requirements to be given to candidate in writing	✓	
Ensure that candidate understands the procedures of all assessment practices.	Appeals Policy Appeals procedure Assessment Policy Assessment Procedure Moderation Policy Moderation procedure Verification Policy Verification Procedure	✓	
Ask the candidate if he/she foresees any problems or identify any special needs.	List needs	✓	

Agreed Assessment Plan

Candidate's Name:	Grant Samson		
Assessor's Name:	Mr NB Dube		
Module name:	Computing Theory		
Unit Standard/s:	❖ PM-07-PS01: Source and compare at least three software toolkits/platforms/ languages used in your field of studies ❖ PM-07-PS02: Identify and contrast four (4) paradigms ❖ PM-07-PS03: Create a programming environment ❖ PM-07-PS04: Write code using a programming language for giving instructions for use ❖ PM-07-PS05: Select and use correct data ❖ PM-07-PS06: Use complex types to organise data ❖ PM-07-PS07: Add API's (Application Programming Interface) to an application ❖ PM-07-PS08: Define a function ❖ PM-07-PS09: Use logical branch statements and comparison operators ❖ PM-07-PS10: Code loops ❖ PM-07-PS11: Use and apply variable scopes ❖ PM-07-PS13: Handle errors and troubleshooting		
Type of Assessment i.e. Formative assignment, Formative test, Formative Practical, Summative etc.	Practical Assessment (PM-07)		
Special Assessment Requirements:			
Event	Date, time and location	Resources required	Evidence to be generated
Assessments due dates.	04/09/2026	Assessments	Completed documentation
Complete activity upload to Campus Online	04/09/2026	Study guide	Completed Portfolio of Evidence
Submit Electronic Portfolio of Evidence on EPOE4ALL	04/09/2026	CampusOnline	Acknowledgement of receipt from Facilitator
Assessor Roles and Responsibility			
Roles	❖ Assessor ❖ Guide ❖ Feedback Agent ❖ Reviewer		
Responsibilities	❖ Consult candidate re-assessment, assessment process and plan.		

	❖ Agree assessment process and plan with candidate. ❖ Forward documentation to candidate: plan, guide and assessment instruments. ❖ Assess candidate with the use of different instruments. ❖ Provide feedback on assessment findings. ❖ Support candidate through assessment process. ❖ Source feedback from candidate on assessment process. ❖ Review assessment process and outcome. ❖ Use assessment process as opportunity to transform assessment activities and outcomes.
--	---

Candidate Roles and Responsibility	
Roles	❖ Learner ❖ Feedback agent ❖ Reviewer
Responsibilities	❖ Be available for assessment. ❖ Be actively involved in the consultative process. ❖ Learn from the assessment process. ❖ Provide feedback to the assessor in terms of the assessment as learning activity. ❖ Provide feedback to the assessor on the efficacy of the assessment process. ❖ Review own role and assessor role in the assessment process.
Assessment Instruments	❖ Electronic Portfolio of Evidence ❖ Questionnaire ❖ Report ❖ Presentation ❖ Reflexive questions ❖ Work sample ❖ Practical's ❖ Group Activity ❖ Research activities
Assessment Process	
❖ Evaluation of E-POE addressing Essential Embedded Knowledge in unit standards. ❖ Evaluation of Research Projects and other evidence addressing specific unit standards. ❖ Consultation: assessment plan and assessment activities and instruments. Pre-assessment moderation and interviews conducted at this stage. ❖ Observation: feedback on assessment against specific outcomes, critical outcomes in unit standards. ❖ Feedback: to candidate regarding sufficiency of evidence and possible interview to gain supplementary evidence. ❖ Feedback to candidate regarding assessment findings as well as review process.	

Feedback	Written feedback to be given to all stakeholders at the end of the assessment process, as well as verbal feedback to the candidate during assessment activities.
Recording Process	Process and findings to be recorded and submitted for record keeping purposes as well as moderation and verification.
Review Process	The review process is the responsibility of the assessor and the candidate. Joint reviewing will take place after feedback has been given to the candidate.
Right to appeal	The candidate must be advised of the right to appeal.

Resources Required	Assignments E-POE Assessments O'Reilly Guides
--------------------	---

I confirm that:

- ❖ I have been consulted on and have agreed to the training and assessment process as detailed in the assessment guide.
- ❖ I have been advised of my right to appeal against any assessment that is unfair, unreliable, invalid or impracticable.
- ❖ I have read and understood the appeal procedure.
- ❖ I know that assessments may be moderated or verified by an external party.
- ❖ The purpose of the assessment has been clearly explained to me.
- ❖ The criteria have been discussed with me, and I know I will be assessed against these criteria.
- ❖ I know when and where I will be assessed, and I was given fair notice.
- ❖ I know how the assessment will be done, and any other requirements related to the assessment.

Signed: 

Date: 31/08/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: Software Developer
Module Title	Computing Theory
Module Code	COT512
NQF Level	5
Assessment	Practical Module PM-07
Hand Out Date	17/08/2026
Hand in Date	04/09/2026
Total Marks	100
Developer	Kobus Mienie
Internal Moderator	Lerato Tsoehlisi
External Moderator	

Student Name and Surname	Grant Samson
Student Number	20261424

INSTRUCTIONS

- Read each question carefully and consider the mark allocation before answering.
- Ensure you answer all questions.
- For final submission, include the Declaration of Authenticity, Pe-Assessment Document, Assessment document, and all the relevant files required to answer the questions in a zip folder.
- Only a .zip will be allowed for zipped folders
- Font style to be used Calibri (Body) 12pt, unless stated otherwise by a question
- Marks will be deducted for missing or incorrectly named documents

ASSESSMENT OUTCOME

- PM-07-PS01: Source and compare at least three software toolkits/platforms/ languages used in your field of studies
- PM-07-PS02: Identify and contrast four (4) paradigms
- PM-07-PS03: Create a programming environment
- PM-07-PS04: Write code using a programming language for giving instructions for use
- PM-07-PS05: Select and use correct data
- PM-07-PS06: Use complex types to organise data
- PM-07-PS07: Add API's (Application Programming Interface) to an application
- PM-07-PS08: Define a function
- PM-07-PS09: Use logical branch statements and comparison operators
- PM-07-PS10: Code loops
- PM-07-PS11: Use and apply variable scopes
- PM-07-PS13: Handle errors and troubleshooting

QUESTION 1

(100)

Instructions: Read the scenario and answer the following questions.

Library Management System

You have found WIL placement at a local library and quickly noticed that they use a manual book search. Develop a simple Library Management System. This system should allow users to manage books and members of the library. The system will require the implementation of various functionalities, including adding new books, searching for books by title or author, registering new members, and checking out books.

Objectives:

- Using Exceptions: Properly handle errors such as trying to check out a book that is not available or entering invalid data.
- Using Assertions: Use assertions to validate the state of the program at various points, such as ensuring a book's availability status is correctly updated when checked out.
- Using Collections and Generics: Utilize collections such as lists or dictionaries to manage books and members. Generics should be used for flexibility and type safety.
- Using Strings and Regular Expressions: Implement functionality to search for books and members using strings and validate inputs (like email addresses for members) using regular expressions.

You pitched this whole idea to the library manager and got approval, however, the library manager asked for a readme file, in which you will explain how the program works.

Source: Mashile T., (2024)

Project Tasks

Task 1: Create the Book and Member Classes

Develop the following classes for the library management system:

- Book Class
 - o Include properties for Title, Author, ISBN, and IsAvailable.
 - o Implement a method that updates the availability status of a book when it is borrowed or returned.
- Member Class
 - o Include properties for Name, Email, and a collection of borrowed books.
 - o Validate the email address using regular expressions when creating a new member object.
 - o Prevent invalid email addresses from being accepted.

Task 2: Manage Library Collections

Use generic collections to store and manage library data.

- Create a Book List to maintain all books in the library.
- Create a Member List to maintain all registered members.
- Implement functionality to add, remove, and display books and members.

Task 3: Implement Search Functionality

Develop a search feature that enables users to locate books based on specific criteria.

- Allow users to search for books by title or author.
- Apply string manipulation techniques to compare search values.
- Ensure that searches are case-insensitive and display all matching results.

Task 4: Implement the Book Checkout System

Create functionality that allows members to borrow books from the library.

- Verify that a book is available before allowing it to be checked out.
- Update the book's availability status after a successful checkout.
- Use exception handling to manage situations where a requested book is unavailable or invalid operations are attempted.
- Display meaningful messages to inform users of successful or unsuccessful transactions.

Task 5: Implement Assertions and Input Validation

Apply validation techniques to improve the reliability of the application.

- Use assertions to verify that books are correctly checked out and returned.
- Validate user input using regular expressions and exception handling.
- Prevent invalid or incomplete information from being processed by the system.
- Ensure that appropriate error messages are displayed when invalid data is entered.

Additional Requirements

- Design and implement a menu-driven console application that enables users to navigate the library management system.
- Incorporate robust exception handling throughout the application to ensure stable operation.
- Display user-friendly error messages whenever exceptions occur.
- Write clear, well-structured, and readable code.
- Include meaningful comments that explain the purpose and logic of important sections of code.
- Apply appropriate naming conventions for classes, variables, methods, and objects.
- Ensure that the application demonstrates good object-oriented programming principles.

End of Assessment

MARKING RUBRIC

Library Management System (100)		
		Mark Allocation
		6 - 10 0 - 5
1. Using Exceptions (20 Marks)		
Proper Implementation (10 Marks): Assess whether exceptions are used appropriately to handle errors such as unavailable books or invalid data entries.		
Error Messages (5 Marks): Evaluate the clarity and usefulness of the error messages displayed to the user when an exception occurs.		
Error Recovery (5 Marks): Check if the program gracefully recovers from exceptions, allowing the user to continue using the system without crashing or behaving unpredictably.		
2. Using Assertions (15 Marks)		
Correct Use (10 Marks): Determine if assertions are correctly used to check the program's state at critical Marks, ensuring the system works as intended (e.g., a book's availability after being checked out).		
Applicability (5 Marks): Evaluate whether the assertions are meaningful and contribute to the stability and reliability of the application.		
3. Using Collections and Generics (20 Marks)		
Implementation (10 Marks): Assess the use of collections and generics to manage books and members, looking for type safety, flexibility, and the efficient use of data structures.		
Data Handling (10 Marks): Evaluate how well the program manipulates collection data, including adding, removing, and searching elements within collections.		
4. Using Strings and Regular Expressions (20 Marks)		
String Manipulation (10 Marks): Review the implementation of search functionality and any other feature requiring string manipulation for effectiveness and efficiency.		
Regular Expressions (10 Marks): Assess the use of regular expressions for validating inputs, such as email formats, ensuring they are correctly applied and functional.		
5. Code Quality and Documentation (15 Marks)		
Readability and Structure (5 Marks): The code should be well-organized, with meaningful variable names, consistent indentation, and a clear separation of functionalities into methods/classes.		
Comments and Documentation (5 Marks): Inline comments and documentation should clearly explain the purpose and logic of the code, making it easy for another developer to understand and maintain.		
Design and Scalability (5 Marks): Evaluate the overall design of the system, including the use of object-oriented principles and how easily new features (like new book or member services) could be added.		
6. User Interface and Experience (10 Marks)		
Usability (5 Marks): The text-based menu system should be intuitive, with clear instructions on how to perform each action.		
Robustness (5 Marks): The application should handle user inputs gracefully, preventing crashes and guiding the user towards correct usage.		
Total:	/100	

MARK ALLOCATION

Task	Description	Marks
Question 1	Library Management System	100

LEARNING OUTCOMES INTEGRATED QUESTIONS CROSS-MAPPING GRID

LO No.	Learning Outcome Description	Question	Marks
PM-07-PS01	Source and compare at least three software toolkits/platforms/ languages used in your field of studies	1	100
PM-07-PS02	Identify and contrast four (4) paradigms		
PM-07-PS03	Create a programming environment		
PM-07-PS04	Write code using a programming language for giving instructions for use		
PM-07-PS05	Select and use correct data		
PM-07-PS06	Use complex types to organize data		
PM-07-PS07	Add API's (Application Programming Interface) to an application		
PM-07-PS08	Define a function		
PM-07-PS09	Use logical branch statements and comparison operators		
PM-07-PS10	Code loops		
PM-07-PS11	Use and apply variable scopes		
PM-07-PS13	Handle errors and troubleshooting		
Total			100